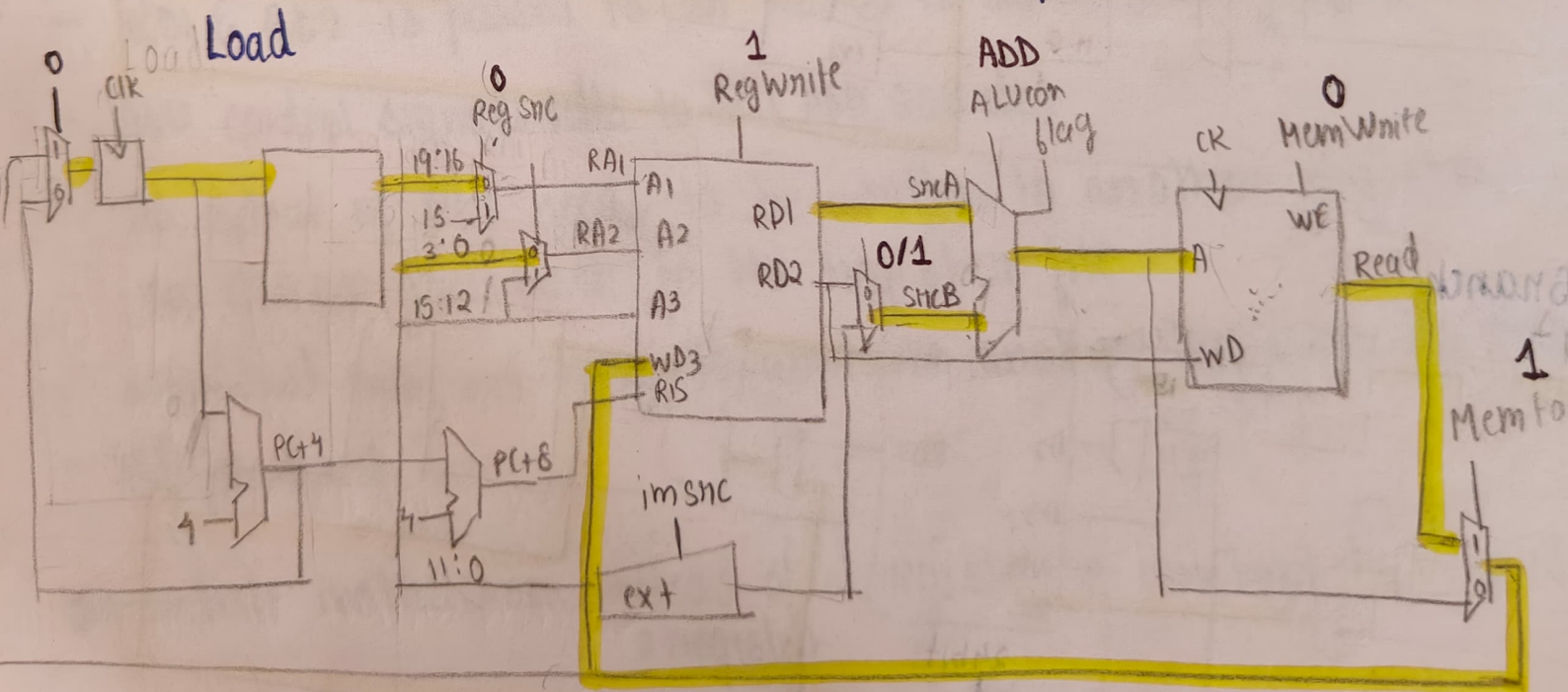


Control signals

- i) PCSnc- determines where the next instruction address comes from.
 $PCSnc = 1$ (Process Jump)
- ii) RegSnc:- selects which instruction fields are used as register address.
- iii) RegWrite:- 0 - no register write STR
 1 - write data to register ADD, LDR
- iv) ALUSnc:- chooses the second operand for ALU
- v) ALU control:- determines which process to perform
- vi) MemWrite :- 0 - do not write memory ADD, LDR
 1 - write data to memory STR
- vii) Mem to reg :- Selects what value written back to register file
- viii) ImmSnc tells extend block how to interpret the immediate bits



LDR R0, [R1], #

- First you need to **Fetch the instruction**, this is done using a **PC (1)**, the address (PC value) is passed to **Instruction memory (4)**
- (4) gives instruction needed to execute
- every cycle **PC (1)** should be **incremented** to execute the instructions in next cycles.
- so (2) is passed to **ALU (3)** to **increment PC by 4 bytes**.
(every instruction is 4 bytes)

----- Instruction Fetch -----

For arithmetic instructions.

- take instruction and **pass source operands to Register file (7)**
- source register numbers are used to read the source registers from the register file.
- **RD1 & RD2** is passed to an **ALU (11)** to do necessary operation
- ALU control signals tells to do (ADD, SUB etc)
- The output of ALU needs to be written to another register so, ALU O/P goes back to the register file
- ~~some inst may not have to write in another register, so mux (12) is used.~~

Immediate instruction instead of a register, a constant is copied to a register.

- ~~RD2~~ is not used, instead a constant is passed
- ~~RD2~~ and **instruction** is passed through **extend** to a **mux (9)** along with RD2.

imm value is initially 16-bit
so sign extend block (8) is used.

Load Instructions

For example: `LDR R0, [R1, #8]`

this is already done by $RD1, RD2, imm$

- ALU is used to add the offset to the value in the base register to find the load address
- we need the data stored in this address so ALU o/p is passed to data memory (11)
- if I want to write back the value at ALU o/p (like ADD instruction) on write back value after reading data from (11) (load inst) is decided using mux (12)

Store Instructions

ex:- `STR R0, [R1, #8]` - already done using ALU

- R0 is ~~used~~ read from $RD2$, this is passed to WD in (11)

Read register (STR) write register (LDR)

Branch Instructions

For ex:- `BEQ R0, R1, offset`

check if equal by $R0 - R1 = 0$

- $R0, R1$ are $RD1$ & $RD2$ resp and passed to ALU done using ALU
- if 0, ALU flag (zero flag set to 1)
- offset is 16-bit val, target address is 32 bit

$$\text{Target addr} = PC + 4 + \text{offset} \times 4$$

- ALU (5) is used for this